ARM ARCHITECTURE

for Embedded Engineers

A practical self-study course from CPU fundamentals to Cortex-M debugging, interrupts, memory, buses, startup code and interview preparation

Focus: ARM Cortex-M and real-world microcontroller firmware

Audience: Embedded engineers, firmware developers and interview candidates

Approach: Concept → architecture → C/assembly → debugging → interview practice

> Keep this book beside your MCU reference manual and debugger.

Contents

1. ARM Ecosystem and Architecture

2. Cortex-M Programmer's Model

3. PC, SP, LR and Function Calls

4. ARM/Thumb Instructions

5. CPU Execution and Pipeline Concepts

6. Memory Architecture and Memory-Mapped I/O

7. Stack, Heap, .data, .bss and Linker Concepts

8. Exceptions and NVIC

9. Interrupt Entry and Context

10. Vector Table and Startup Sequence

11. AHB, APB and AXI

12. DMA and Peripheral Transfers

13. volatile, Atomicity, Barriers and Critical Sections

14. MPU, Cache and Performance

15. Faults and Debugging

16. Cortex-M vs Cortex-A/R

17. How to Read an MCU Reference Manual

18. Practical Debugging Workflow

19. Hands-On Exercises

20. Embedded Interview Questions

21. Final Revision Checklist

Appendix: Cheat Sheet

How to study

• Learn the programmer's model before memorizing instructions.

• Connect every concept to a debugger observation: PC, SP, LR, memory, stack or interrupt.

• Use one real MCU throughout the course and map the generic architecture to its reference manual.

• After each chapter, solve the questions without looking at the answer.

Chapter 1 — ARM Ecosystem and Architecture

Architecture vs core vs MCU

ARM architecture defines instruction, register, memory and exception rules. Cortex-M is a family of ARM M-profile processor cores. A microcontroller such as an STM32 or Renesas RA combines a CPU core with flash, SRAM, clocks and peripherals.

The three-layer model

```
ARM Architecture
       |
       v
Cortex-M CPU core
       |
       v
MCU implementation
       |
       +-- Flash
       +-- SRAM
       +-- GPIO / UART / SPI / I2C
       +-- Timers / ADC / DMA
```

ARM profiles

Profile	Typical use	Examples
A	Application processors / rich OS	Cortex-A53
R	Real-time systems	Cortex-R5
M	Microcontrollers / RTOS / bare metal	Cortex-M4, M7, M33

Key idea

For embedded firmware, first master Cortex-M. Later, the same mental model can be extended toward Cortex-A and embedded Linux.

Chapter 2 — Cortex-M Programmer's Model

Core registers

Register	Purpose
R0–R3	General purpose; common argument/return registers
R4–R11	General purpose; commonly preserved across calls
R12	Scratch register
SP/R13	Stack Pointer
LR/R14	Link Register
PC/R15	Program Counter
xPSR	Processor status and execution state

PC

PC represents the execution location. Branches, function calls, exception entry and returns alter the instruction flow.

SP

SP identifies the current stack position. Function state, local storage and exception frames use the stack.

LR

LR stores the normal subroutine return path. During exception handling, LR can contain special exception-return information.

xPSR

```
N = Negative
Z = Zero
C = Carry
V = Overflow

CMP R0, R1
BEQ equal_label
```

Interview checkpoint

Explain PC, SP and LR without saying only 'they are registers'. Explain what hardware behavior each one participates in.

Chapter 3 — PC, SP, LR and Function Calls

Function-call model

```
main()
  |
  | BL function
  v
function()
  |
  | work
  |
  | return
  v
main()
```

BL

Branch with Link changes control flow and preserves a return path in LR. A function may save LR on the stack when it needs to make further calls or preserve it across its own work.

Typical prologue/epilogue

```
function:
    PUSH {R4, LR}
    ...
    POP  {R4, PC}
```

Stack frame

```
Higher addresses
+------------------+
| caller state     |
+------------------+
| saved registers  |
+------------------+
| local variables  |
+------------------+
| temporary space  | <- SP
+------------------+
Lower addresses
```

Debugging value

Understanding stack frames explains debugger backtraces, recursion, corrupted return addresses and many HardFaults.

Chapter 4 — ARM/Thumb Instructions

Essential instructions

Instruction	Meaning
MOV	Move value/register
ADD / SUB	Arithmetic
CMP	Compare and update flags
LDR / STR	Load/store memory
B	Branch
BL	Branch with link / call
BX	Branch to register
PUSH / POP	Save/restore stack state
MRS / MSR	Access special registers

C to assembly

```
C:
int c = a + b;

Conceptual assembly:
LDR R0, [a_address]
LDR R1, [b_address]
ADD R2, R0, R1
STR R2, [c_address]
```

Important distinction

ADD operates on registers. LDR/STR connect registers to memory. This distinction is fundamental to understanding assembly and memory-mapped peripherals.

Practice

Compile a simple C function with an ARM compiler and inspect the generated disassembly. Compare the optimized and unoptimized versions.

Chapter 5 — CPU Execution and Pipeline Concepts

Fetch → decode → execute

```
Fetch instruction
      |
      v
Decode operation
      |
      v
Execute
      |
      v
Write result / continue
```

Pipeline

Real Cortex-M cores use pipelines to improve throughput. The number of stages and exact timing vary by core. Branches, memory wait states and exceptions can disrupt normal flow.

Cycles are not instructions

One instruction does not necessarily mean one CPU cycle. Flash wait states, memory system behavior, bus contention and branches can change execution time.

Interview question

Why can the same code execute at different speeds from different memory regions? Because instruction fetch and data access latency can differ, and some regions may require wait states or acceleration mechanisms.

Chapter 6 — Memory Architecture and Memory-Mapped I/O

Conceptual memory map

```
0xFFFFFFFF  System / implementation
0xE0000000  System control / NVIC
0x60000000  External device space
0x40000000  Peripheral space
0x20000000  SRAM
0x00000000  Code / Flash region
```

Vendor-specific addresses

The exact flash, SRAM and peripheral addresses belong to the MCU implementation. Never assume a base address from another MCU family.

Memory-mapped I/O

```
#define GPIO_REG (*(volatile unsigned int *)0x40020014u)

GPIO_REG |= (1u << 5);
```

volatile

Hardware registers can change independently of the current instruction stream. volatile tells the compiler that accesses are observable side effects. It does not create atomicity or a lock.

Endianness

In little-endian storage, a 32-bit value such as 0x12345678 is stored with 0x78 at the lowest byte address.

Chapter 7 — Stack, Heap, Sections and Linker

Sections

Section	Typical contents
.text	Program code
.rodata	Constants / strings
.data	Initialized writable globals/statics
.bss	Zero-initialized globals/statics
stack	Automatic variables / saved context
heap	Dynamic allocation

Startup initialization

```
Flash image
  .text
  .rodata
  initial .data values

RAM after startup
  .data  <- copied from Flash
  .bss   <- cleared to zero
  heap
  stack
```

Linker script

The linker maps sections to physical memory regions. This controls where code, data, bootloader images and application slots live.

Example memory definition

```
MEMORY
{
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 1024K
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}
```

Bootloader relevance

A custom bootloader/application partition must match the linker layout. A correct C program can still fail if its sections are linked outside the actual image region.

Chapter 8 — Exceptions and NVIC

Exception model

Cortex-M uses exceptions for reset, faults, system services, SysTick and external interrupts.

Important exceptions

Exception	Purpose
Reset	Starts execution
NMI	Non-maskable interrupt
HardFault	Serious fault / escalation
MemManage	Memory protection fault when supported
BusFault	Bus access fault when supported
UsageFault	Execution-related fault
SVCall	Supervisor call
PendSV	Often used by RTOS context switching
SysTick	System timer
External IRQ	Peripheral interrupt

NVIC

The Nested Vectored Interrupt Controller manages external interrupt enable/pending/active state and programmable priority.

Nested interrupts

A sufficiently high-priority interrupt can preempt a lower-priority active handler. Priority details depend on the core and MCU implementation.

Chapter 9 — Interrupt Entry, Exit and Context

Interrupt flow

```
Peripheral event
      |
      v
NVIC pending
      |
      v
CPU accepts exception
      |
      v
Hardware stacks context
      |
      v
Vector lookup
      |
      v
ISR executes
      |
      v
Exception return
      |
      v
Interrupted code resumes
```

Standard exception frame

For a standard Cortex-M exception entry, hardware stacks R0–R3, R12, LR, PC and xPSR. Floating-point context can add more state depending on configuration.

Why stacked PC matters

The stacked PC gives a starting point for identifying where execution was interrupted or where a fault occurred. Combined with fault status registers, it is a powerful debugging clue.

RTOS connection

PendSV is commonly used for context switching because it can be assigned a low priority and run after higher-priority interrupts have been serviced.

Chapter 10 — Vector Table and Startup

Vector table

```
+0x00  Initial Stack Pointer
+0x04  Reset_Handler
+0x08  NMI_Handler
+0x0C  HardFault_Handler
+0x10  MemManage_Handler
+0x14  BusFault_Handler
...
```

Reset sequence

```
Power-on / Reset
      |
      v
Read initial SP
      |
      v
Read Reset_Handler
      |
      v
Startup code
      |
      +--> clocks / system init
      +--> .data copy
      +--> .bss clear
      |
      v
main()
```

Bootloader handoff

A robust application jump must account for the application's initial stack pointer, vector-table location and interrupt state, not just set PC to an application address.

Interview checkpoint

Explain why the vector table is essential: it maps reset and exception numbers to actual handler entry addresses.

Chapter 11 — AHB, APB and AXI

Why buses exist

The CPU, DMA, memory and peripherals need an interconnect for transactions. AMBA protocols organize this communication.

AHB

A higher-performance AMBA bus commonly used for system/memory communication in MCU designs.

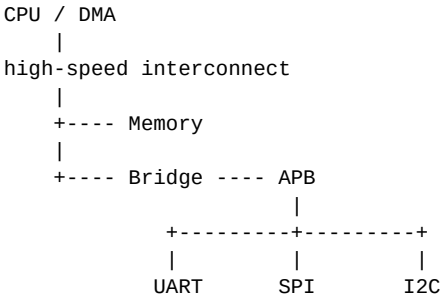
APB

A simpler peripheral bus commonly used for lower-bandwidth blocks such as UART, timers and I2C.

AXI

A higher-performance AMBA protocol common in more complex SoCs and Cortex-A systems.

Conceptual topology



Important

The actual topology is implementation-specific. Always use the MCU block diagram and reference manual.

Chapter 12 — DMA and Peripheral Transfers

CPU-driven transfer

```
for (i = 0; i < N; i++)
    destination[i] = source[i];
```

DMA-driven transfer

```
CPU configures:
  source
  destination
  length
  trigger

DMA moves data
DMA optionally raises interrupt
CPU handles completion
```

Why DMA

DMA reduces CPU overhead and is useful for ADC streams, UART/SPI buffers, memory copies and high-rate peripheral traffic.

DMA debugging

Check source/destination addresses, transfer length, trigger source, bus permissions, alignment, interrupt status and buffer ownership.

Cache warning

On cache-enabled systems, CPU cache contents can differ from RAM. DMA buffers may require cache clean/invalidate operations or non-cacheable placement.

Chapter 13 — volatile, Atomicity, Barriers and Critical Sections

volatile

```
volatile uint32_t status;

while ((status & RX_READY) == 0) {
}
```

What volatile does not do

volatile does not make a read-modify-write sequence atomic, does not protect shared data and does not replace an RTOS mutex.

Atomicity

```
reg |= MASK;

Conceptually:
temp = reg;
temp = temp | MASK;
reg = temp;

Another execution context can interfere between steps.
```

Critical sections

Use the smallest appropriate critical section. The correct mechanism depends on whether the competing context is an ISR, another task, another CPU or DMA.

Barriers

DMB, DSB and ISB have different architectural purposes. Use CMSIS intrinsics and the ARM documentation rather than treating all barriers as equivalent.

Chapter 14 — MPU, Cache and Performance

MPU

The Memory Protection Unit can define memory regions and access permissions. It is useful for isolating tasks, protecting stacks and detecting invalid accesses where supported.

Cache

Caches improve average access time but add complexity around misses, maintenance and DMA coherency.

Performance checklist

Check compiler optimization, flash wait states, memory alignment, bus contention, DMA opportunities and measured timing before optimizing.

Advanced point

A faster CPU clock does not automatically make a peripheral transaction faster. Peripheral clocks, protocol timing and external device limits can dominate.

Chapter 15 — Faults and Debugging

Common causes

Invalid pointers, stack overflow, corrupted return addresses, invalid instruction flow, bus errors, incorrect vector configuration, race conditions and hardware/clock problems.

HardFault workflow

```
1. Stop in HardFault_Handler
2. Capture stacked PC
3. Capture stacked LR and xPSR
4. Read fault-status registers
5. Map PC to source/assembly
6. Inspect SP and stack memory
7. Check recent pointer/interrupt/stack changes
```

Debugger questions

• What is PC?

• Is SP inside valid RAM?

• Does LR make sense?

• Which instruction faulted?

• Which memory address was accessed?

• Was an interrupt active?

• Was the stack close to its limit?

Example

```
PC  = 0x08013A42
SP  = 0x20007F00
LR  = 0xFFFFFFF9

Start with the instruction at PC.
Inspect the exception frame at SP.
Treat the special LR value as exception-return information.
```

Practical rule

Do not reset immediately after a fault while debugging. First preserve the processor state that explains the failure.

Chapter 16 — Cortex-M vs Cortex-A vs Cortex-R

Comparison

Feature	Cortex-M	Cortex-A	Cortex-R
Typical role	MCU / deeply embedded	Application processor	Real-time systems
Software	Bare metal / RTOS	Linux / rich OS	RTOS / safety
MMU	Usually absent	Common	Depends on core
MPU	Common on many models	Different memory model	Common in many designs
Typical complexity	Lower	High	Medium/high

Embedded Linux connection

When you move from Cortex-M to Cortex-A, PC/SP/register concepts remain, but MMU, caches, privilege levels, virtual memory and OS scheduling become much more prominent.

Chapter 17 — How to Read an MCU Reference Manual

Recommended order

Step	Find this first
1	CPU/core and memory map
2	Reset and clock architecture
3	Interrupt/vector information
4	Peripheral chapter
5	Register summary
6	Timing/electrical details
7	Errata

UART investigation

Find base address, peripheral clock, reset/enable sequence, pin multiplexing, register offsets, interrupt number, baud-rate calculation, status flags and errata.

Register discipline

```
Peripheral Base
  +0x00 CONTROL
  +0x04 STATUS
  +0x08 DATA
  +0x0C BAUD
  +0x10 INTERRUPT_ENABLE
```

Never assume

Read reset values, access type, side effects and clear semantics. Names such as STATUS or ENABLE are not enough to infer behavior.

Chapter 18 — Practical Debugging Workflow

Firmware bring-up

```
1. Confirm power/reset
2. Confirm debugger connection
3. Inspect CPU/core
4. Check PC and SP
5. Verify vector table
6. Verify clock setup
7. Verify memory initialization
8. Test GPIO
9. Add one peripheral at a time
10. Add trace/logging where useful
```

Does not reach main()

• Check initial SP.

• Check Reset_Handler address.

• Check flash contents.

• Check startup code.

• Check clocks and watchdog.

• Check linker placement.

• Check vector-table relocation.

Peripheral register write ineffective

• Peripheral clock enabled?

• Peripheral out of reset?

• Correct register address?

• Correct pin mux?

• Write protection?

• Correct access width?

• Correct volatile declaration?

• Correct clear/write semantics?

Chapter 19 — Hands-On Exercises

Solve these before reading the answer key.

Level 1 — Programmer model

1. What is the difference between PC and SP?

2. Why is LR important during a function call?

3. What happens to Z after CMP R0,R1 when both contain 25?

4. Which registers commonly carry the first function arguments?

5. Why is xPSR useful during debugging?

Level 2 — Memory and linker

6. Why are peripheral registers memory-mapped?

7. What is the difference between .data and .bss?

8. Why can a wrong linker script break a bootloader?

9. What does volatile solve and what does it not solve?

10. Why must peripheral addresses be taken from the specific MCU manual?

Level 3 — Interrupts

11. What does NVIC do?

12. What is normally stacked on Cortex-M exception entry?

13. How does interrupt priority enable nesting?

14. What is the purpose of the vector table?

15. Why is stacked PC valuable during HardFault analysis?

Level 4 — Debugging scenarios

16. The CPU resets before main(). List five checks.

17. UART interrupt never fires. Give a systematic sequence.

18. A task crashes after hours. What stack/heap/concurrency checks do you perform?

19. DMA returns old data on a cache-enabled core. What do you investigate?

20. An app works standalone but fails after a bootloader jump. What do you check?

Mini practical lab

Compile a small ARM C program, inspect its .map file and disassembly, locate main(), a function call and the stack prologue/epilogue, then single-step while watching PC, SP and LR.

Chapter 20 — Embedded Interview Questions and Model Answers

What is the difference between ARM architecture and Cortex-M?

Model answer: Architecture defines rules and behavior; Cortex-M is a family of processor cores implementing the ARM M-profile.

What is PC?

Model answer: The Program Counter represents the instruction execution location and changes as control flow changes.

What is SP?

Model answer: The Stack Pointer identifies the current stack position used for function and exception state.

What is LR?

Model answer: The Link Register holds a normal subroutine return path and can contain special exception-return information.

What is memory-mapped I/O?

Model answer: Peripheral registers occupy addresses in the processor address space so normal load/store operations can control hardware.

What is NVIC?

Model answer: The Cortex-M interrupt controller that manages external interrupt enable, pending/active state and priority.

What happens on interrupt entry?

Model answer: The processor accepts the exception, stacks an exception frame, vectors to the handler and later performs exception return.

What is the vector table?

Model answer: A table containing the initial stack pointer and exception/interrupt handler addresses.

Why use volatile?

Model answer: To preserve required observable accesses to memory-mapped or externally changing objects. It does not make code atomic or synchronized.

What is HardFault?

Model answer: A serious Cortex-M fault exception that can occur directly or through escalation of other faults.

Stack vs heap?

Model answer: The stack supports automatic execution context through SP; the heap supports dynamic allocation and is managed by an allocator/runtime.

What is DMA?

Model answer: Hardware that transfers data between memory and peripherals, or between memory regions, with reduced CPU involvement.

AHB vs APB?

Model answer: Both are AMBA protocols; AHB targets higher-performance system communication while APB is a simpler peripheral bus.

Why does a linker script matter?

Model answer: It places program sections into physical memory and therefore controls image layout, RAM usage and bootloader/application placement.

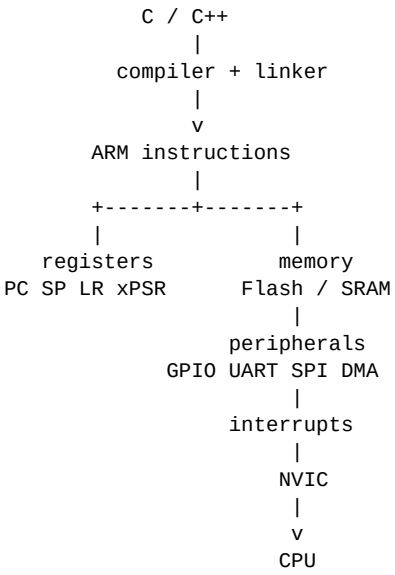
Why can a bootloader jump fail?

Model answer: The application may require the correct initial SP, vector-table location, interrupt state, clocks and memory layout; setting only PC is not always sufficient.

Chapter 21 — Final Revision Checklist

Area	You should be able to explain
Registers	R0–R12, SP, LR, PC, xPSR
Control flow	B, BL, BX, function calls and returns
Stack	Frames, growth, overflow, exception context
Memory	Flash, SRAM, peripherals, memory map
C runtime	.text, .rodata, .data, .bss, stack, heap
Linker	Memory regions and section placement
Interrupts	Vector table, NVIC, priority, nesting
Exceptions	Reset, HardFault, SysTick, PendSV
Buses	AHB, APB, AXI concepts
DMA	Hardware data movement and ownership
Concurrency	volatile, atomicity, critical sections
Debugging	PC/SP/LR, stacked frame, fault registers
Startup	Reset_Handler through main()
Reference manual	Memory map, registers, clocks, interrupts, errata

Final mental model



If you can trace a problem from C code → assembly → register → memory/bus → peripheral → interrupt → CPU context, you have the core mental model of ARM-based embedded firmware.

Appendix — Quick ARM Cheat Sheet

Item	Remember
PC	Execution location / control flow
SP	Current stack position
LR	Subroutine return path / exception-return information
xPSR	Processor status and execution state
LDR	Memory → register
STR	Register → memory
BL	Call / branch with link
BX	Branch to register
NVIC	Cortex-M interrupt controller
Vector table	Initial SP + handler addresses
.text	Code
.rodata	Constants
.data	Initialized writable data
.bss	Zero-initialized data
AHB	Higher-performance AMBA system bus
APB	Simple peripheral bus
DMA	Hardware data movement
volatile	Observable accesses; not synchronization
HardFault	Serious fault exception
Linker script	Places sections into memory

Suggested study project

Choose one ARM MCU you use in your work—such as a Renesas RA or STM32 Cortex-M device—and reproduce this book's concepts against its actual reference manual. Find its vector table, memory map, clock tree, UART registers, NVIC interrupt number, linker memory regions and startup file. This converts architecture knowledge into practical engineering skill.